Society for Computer Technology & Research's

PUNE INSTITUTE OF COMPUTER TECHNOLOGY, Pune.

DEPARTMENT OF INFORMATION TECHNOLOGY

CIE 2 – CASE STUDY-BASED DATA STRUCTURES ACTIVITY

ONE-PAGE GROUP REPORT

Class: SY-11 Division: SY-11

Practical Batch: H-11
Group Members (Roll No. and Name):

1. 23357 – Dhiraj Yenkure
2. 23358 – Aniruddha Kaje
3. 23365 – Bhushan Karvande
4. 23368 – Arman Shaikh

Topic/Case Study Title: College Club Membership Management System Using Linked Lists and Hashing

Problem statement:

In a college, many clubs maintain their own member lists using student/member IDs. The system must create clubs, add, remove, search and display members, reject duplicate memberships, and compare two clubs to find all unique members (union), common members (intersection) and members exclusive to each club. The program solves this using singly linked lists to store clubs and members, and a manually implemented hash table with separate chaining (table size 101) for fast membership checking. It is menu-driven and also displays membership statistics (total clubs, total memberships and the largest club).

Objectives:

1. To create and manage clubs and their members (add, remove, search, display) using singly linked lists.
2. To implement a manual hash table with separate chaining for fast membership search and duplicate prevention.
3. To compare two clubs (union, intersection, only-in-each-club) and display membership statistics.

Data structures:

1. Singly Linked List: The ll class stores the member IDs of each club in insertion order (with head and tail pointers for O(1) insertion at the end, and a count of members). The clubs themselves are also kept in a singly linked list through the next pointer of each Club object. Each member node (snode) holds an ID and a pointer to the next node.
2. Hash Table with Separate Chaining: Each club has its own hash table (array of 101 snode pointers) that uses the hash function (id % 101) to map member IDs to indices. Collisions are handled by linked-list chains, giving O(1) average search, insertion and deletion. It is used to reject duplicate members and, in Compare Two Clubs, temporary hash tables remove duplicates while building the union, intersection and exclusive-member lists.
3. Class-based design: Club (club ID, name, member list, member hash table) and ClubManager (linked list of clubs; create club, display clubs, add, remove, search and display members, compare clubs, statistics) with a menu-driven interface in main().

Implementation:

Programming language: C++

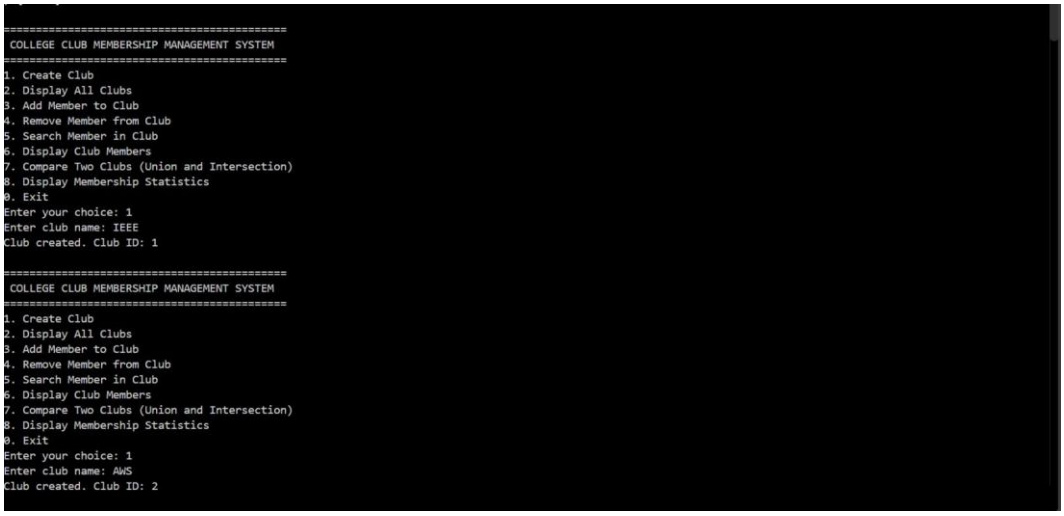
Tools: Visual Studio Code with a C++ compiler (g++), run in the terminal.

Methods implemented: Manual hashing (separate chaining) for member search, duplicate prevention, union, intersection and exclusive-member detection (only in Club A / only in Club B); linked-list insertion, deletion and display for members; no STL containers used for storage (only string for names).

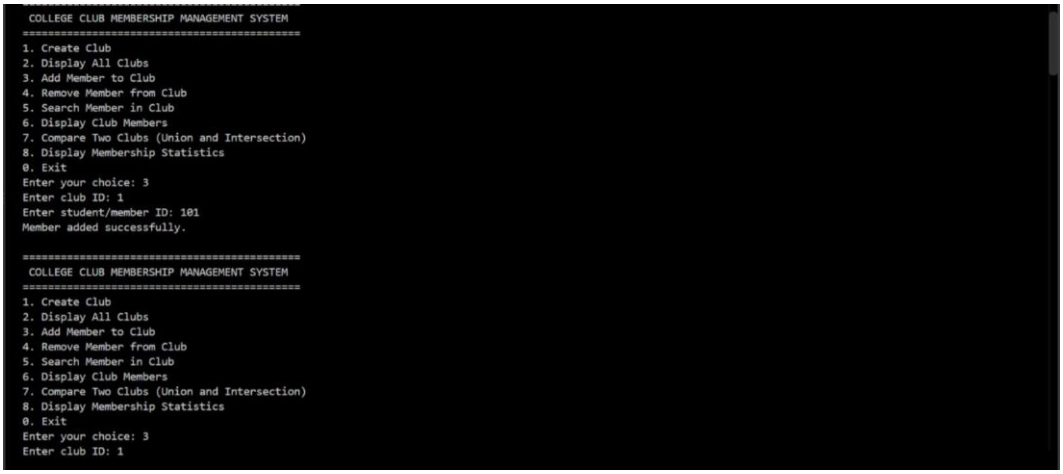
GitHub Repository Link: https://github.com/aniruddha1127/college-club-membership-management

Implementation output:

Output 1: Create Club option – clubs IEEE (ID 1) and AWS (ID 2) created.



Output 2: Add Member to Club – member 101 added to IEEE (club ID 1).



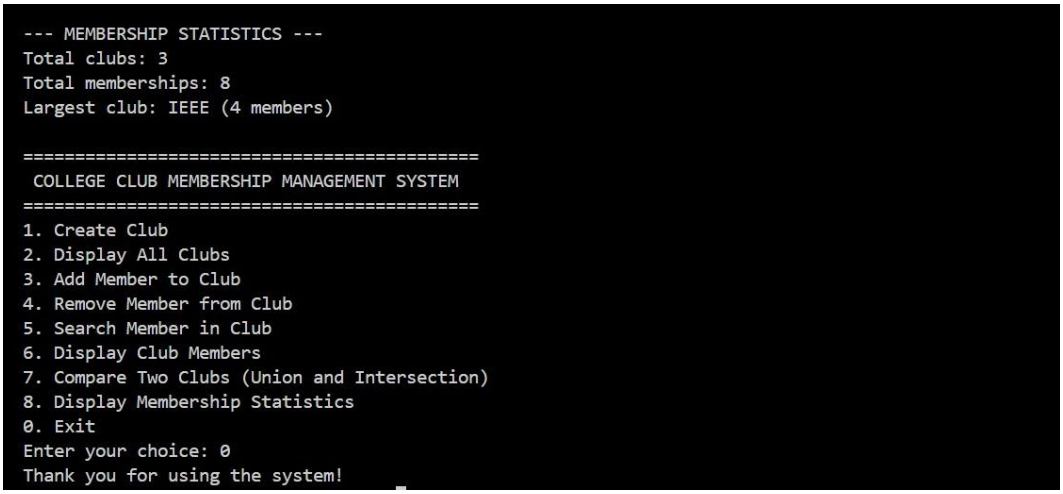
Output 3: Display All Clubs – IEEE (4 members), AWS (3 members), ACM (1 member).



Output 4: Compare Two Clubs (AWS and IEEE): union, intersection and exclusive members, followed by Membership Statistics.



Output 5: Membership Statistics (3 clubs, 8 memberships, largest club IEEE) and Exit.



Conclusion / learning key outcome:

The project demonstrates how singly linked lists and manually implemented hash tables with separate chaining can be used to manage college club memberships. Hashing gives fast duplicate checking and member search, while linked lists give simple, dynamic storage with O(1) insertion at the end, and together they make comparing two clubs (union, intersection and exclusive members) efficient and easy to understand.

Student signatures

1. _____

2. _____

3. _____

4. _____

Faculty remarks: